Project Proposal - Passakorn Sukhawvadeekul

Simplified Distributed Key-Value Store

As Internet usage continues to grow, online services must accommodate increasing workloads while maintaining availability and reliability. However, a monolithic system running on a single server can become a performance bottleneck and a single point of failure. Although upgrading hardware can increase capacity, it does not eliminate the risk of server failure or provide unlimited scalability. Therefore, distributed systems offer an alternative approach by distributing workloads across multiple machines to improve scalability and fault tolerance.

Distributed systems use techniques such as data partitioning and replication to improve scalability and availability. In *Dynamo: Amazon's Highly Available Key-value Store*, DeCandia et al. (2007) describe a distributed storage system that uses consistent hashing, replication, and quorum-based operations to handle node failures while maintaining high availability. These techniques provide a foundation for understanding the benefits and trade-offs of distributed key-value stores compared with monolithic systems.

This project proposes to implement and compare a monolithic key-value store running on a single server with a distributed key-value store inspired by Dynamo's architecture. Both systems will support basic key-value operations, allowing their performance and reliability to be evaluated under comparable workloads. Experiments will examine how the systems handle increasing numbers of client requests and how they behave when a server or storage node becomes unavailable. Performance will be evaluated using throughput, response latency, and request success rate. The results will be used to compare the scalability, availability, and fault tolerance of the two approaches and to identify the trade-offs involved in designing distributed storage systems.

Project Timeline & Due Dates

Week 1-2: Literature Review and System Design

Read both assigned papers, review essential distributed systems concepts, define project requirements, design both architectures, and establish evaluation metrics.

Week 3: Monolithic Implementation

Implement a single-node key-value store supporting PUT, GET, and DELETE, establish a performance baseline.

Week 4: Distributed Implementation I

Implement independent storage nodes, inter-node communication, and consistent hashing for key distribution.

Week 5: Distributed Implementation II

Implement replication, request routing, and basic failure handling. Test the system when a node becomes unavailable.

Week 6: Testing and Experiments

Compare both systems under increasing client workloads. Simulate server and node failures. Measure throughput, latency, and request success rate.

Week 7: Analysis and Report Writing

Analyze the results, compare the architectures, discuss limitations, and complete the written report.

Week 8: Finalization and Presentation

Fix critical bugs, verify experimental results, document the source code, finalize the report, and prepare and rehearse the presentation.

Due Date: Final Submission by the end of Week 8-10

Project Deliverable

Functional Systems: a monolithic key-value store and a simplified distributed key-value store, both supporting read and write operations.

Written Report: A comprehensive report of at least four pages detailing the project's objectives, methodology, implementation, experimental results, and conclusions.

Presentation: A 15-minute video presentation accompanied by slides, highlighting the project's objectives, system architecture, implementation, experimental results, and key findings.